Software Design Description (SDD)

Project: Pathfinder

Version: 1.0

Authors: Patrick King, Maddie McDowell, Nic Ortiz, Thomas Powell

1. Introduction

1.1 Purpose

This document defines the high-level software architecture and design decisions for *Pathfinder*, a career development web application. It outlines system components, their responsibilities, and interactions to ensure consistent implementation, scalability, and maintainability.

1.2 Scope

Pathfinder is a React/Node.js-based web application with AI-driven career guidance features. It helps users explore career paths, improve resumes, find jobs, and connect with learning resources and professionals.

2. System Overview

2.1 Architecture Pattern

The system follows a **modular, client-server architecture** using the **MVC (Model-View-Controller)** paradigm:

Frontend: Built with React and responsive CSS
Backend: Built with Node.js, integrates OpenAI and Firebase
Database: Firebase (Cloud Firestore and Authentication)

External APIs: OpenAI, LinkedIn, Job Boards

3. System Components

3.1 Front-End Components

Component	Description
Landing Page	Login/Signup with responsive layout
Dashboard	Centralized user hub displaying personalized content
Profile Setup Page	Allows input of career data and preferences
Chatbot UI	Floating AI assistant available on all pages

3.2 Back-End Components

Component	Description
Auth & User Management	Firebase Authentication and access control
Career Guidance Engine	Processes questionnaire results to suggest careers
Resume & Job Matching	Analyzes resumes and fetches jobs based on criteria
External API Integration	Connects to job boards, LinkedIn, and learning platforms

3.3 AI Chatbot

Subcomponent	Function
Query Router	Sends user queries to OpenAI for interpretation
Context Handler	Maintains session data for natural conversation
Resource Linker	Adds external guidance suggestions to conversations

3.4 Database Design

Users: Authentication info, profile data
Preferences: Career choices, goals, skill levels
Resumes: Uploaded documents, analysis results

History: User activity, interactions

4. System Interfaces

4.1 External Interfaces

Interface	Purpose
Job Board APIs	Fetch job listings relevant to user interests
LinkedIn API	Retrieve networking and mentorship opportunities
OpenAI API	Power AI chatbot and resume feedback features

4.2 Internal Interfaces

React ↔ Node.js: JSON-based RESTful APIs
Node.js ↔ Firebase: Firebase SDK for CRUD operations

5. Data Flow & Control Flow

User logs in → Profile loaded → Dashboard presented
Chatbot available → Queries routed → OpenAI generates response
Resume uploaded → Analyzed → Jobs matched
Job board queried → Matches stored → Results shown on dashboard

6. Design Constraints

Must use React, Node.js, Firebase, and OpenAI APIs
Must support accessibility (screen readers, keyboard navigation)
Must run efficiently on mobile and desktop

7. Security Design

Passwords encrypted using Firebase Auth
HTTPS enforced
Data access restricted to authenticated users only

8. Deployment Architecture

Frontend deployed via Node.js
Backend hosted on Firebase
Firebase as persistent data layer

9. Quality Attributes

Attribute	Implementation Strategy
Usability	Intuitive UI, built-in onboarding tutorial
Performance	Lightweight UI; optimized queries and API calls
Scalability	Stateless backend functions; cloud-hosted services
Availability	Cloud-based deployment with redundancy
Maintainability	Modular components with clean separation of concerns

10. Appendices

Glossary:
API: Application Programming Interface
UI: User Interface
SRS: Software Requirements Specification
Revision History: TBD during iterative development